

SDPAR Kernel V3 — Полный комплекс с учетом всех нюансов

ID журнала без владельца: 267d28ef2a4e2f1e / 2026-09-01 Copenhagen / Christiania / Подписано: живой человек

Gist: <https://gist.github.com/nablydatel8-cell/5cb83a678bb1ea3b5517139768e4e34e> (04.09.2026, 2 файла, public, по договору держится)

Фиксация: Копенгаген 1 сентября 2026, MIT 2026, Halsnæs Kommune, 5°C Skyet, 11:00, ThinkCentre + HP клавиатура

Источник: записная книжка «Прозктидея» — фундаментальная основа всей архитектуры

1. Что такое V3 — «Тетрадь которую нельзя стереть» — зрелая формулировка

Изначальная формулировка «нельзя удалить технически и юридически» — абсолютистская, уязвимая для критики.

Зрелая инженерная правка (Gemini, 04-09-2026):

Переход к принципу обнаруживаемости изменений (tamper-evident) — это водораздел между иллюзией безопасности и реальной криптографической инженерией.

Мы НЕ утверждаем, что файл нельзя удалить. Мы гарантируем: любое несанкционированное вмешательство становится математически очевидным при `verify_full_integrity()`.

2. Схема БД — две исторические версии

2.1 Простая версия (из первого README):

sql

```
1 CREATE TABLE IF NOT EXISTS audit_log (  
1   sequence INTEGER PRIMARY KEY AUTOINCREMENT,  
1   hypothesis_id TEXT NOT NULL,  
1   prediction_hash TEXT NOT NULL,  
1   observation_hash TEXT NOT NULL,  
1   evaluation_result TEXT NOT NULL,  
1   prev_hash TEXT,  
1   entry_hash TEXT NOT NULL  
1 );  
1 CREATE TRIGGER trg_prevent_update BEFORE UPDATE ON audit_log BEGIN SE  
1 CREATE TRIGGER trg_prevent_delete BEFORE DELETE ON audit_log BEGIN SE
```

```
PRAGMA journal_mode=WAL + INDEX idx_hypothesis_id
```

2.2 Боевая PUBLIC clean версия (из PUBLIC/SDPARKernelV3.py, со скринов 04.09 14:23):

sql

```
1 CREATE TABLE IF NOT EXISTS audit_log (  
1   sequence INTEGER PRIMARY KEY,  
1   state_version INTEGER,  
1   timestamp REAL,  
1   hypothesis_id TEXT,  
1   observation_data TEXT,  
1   evaluation_result TEXT,  
1   prev_hash TEXT,  
1   event_hash TEXT,  
1   key_id TEXT,  
1   signature_hex TEXT  
1 );  
1 CREATE TABLE IF NOT EXISTS state_meta (key TEXT PRIMARY KEY, value TE  
1 INSERT OR IGNORE INTO state_meta (key,value) VALUES ('version','0')
```

Здесь `sequence` — не `AUTOINCREMENT`, а вычисляется в транзакции,
`event_hash == entry_hash` по смыслу, добавлены `state_version` ,
`timestamp` , `key_id` , `signature_hex` .

3. Защита от стирания — нюанс

Что делают триггеры:

- `trg_prevent_update / trg_prevent_delete` → `RAISE(ABORT, 'append-only')`
- Защищают от штатных манипуляций на уровне SQL (UPDATE/DELETE через SQLite API).

Чего НЕ делают:

- НЕ защищают от прямой подмены файла базы тем, кто имеет физический или административный доступ к дисковому хранилищу.
- Поэтому триггеры — это только первый контур. Второй контур — хэш-цепь + внешняя публикация.

Ошибка пользователя на этапе публикации:

Вставка raw MIME письма в gist (`Delivered-To` , `ARC-Seal` , `Content-Type: Text/html; charset=UTF-8` , `Content-Transfer-Encoding: base64` , строки `v9GA0L70stC10Y...` , `PGRpd17Q...`) — ломает gist. Правильно — вставлять только тело письма от `# SDPARKernelV3` до `Commit: nablydatel8@gmail.com` .

4. Хэш-цепь доверия

- `GENESIS = "0"*64`
- Логика в `process_and_commit()` :

python

```
1 cursor.execute("SELECT sequence, event_hash FROM audit_log ORDER BY s
1 row = cursor.fetchone()
1 prev_hash = row[1] if row else "0"*64
```

```
1 next_seq = (row[0] + 1) if row else 1
```

- `envelope = {sequence, state_version, timestamp, hypothesis_id, observation_data, evaluation_result, prev_hash}`
- `canonical_bytes = json.dumps(envelope, sort_keys=True, separators=(',', ':'), ensure_ascii=False).encode('utf-8')` — `canonical_hash(sort_keys=True)`
- `event_hash = sha256(canonical_bytes).hexdigest()`
- Цепь: `prev_hash` -> `event_hash` от GENESIS.

Любая попытка вставить запись задним числом ломает `prev_hash` → ломается весь последующий `event_hash`.

5. Верификация — автономная

`verify_full_integrity()`:

- Пересчитывает всю цепь от GENESIS через `canonical_hash`
- Ловит `Broken chain / Tampering detected`
- В PUBLIC версии еще проверка подписей и `state_version` монотонности.
- Публичная верификация — любой может вызвать, без приватного ключа.

6. Конкурентность и версионирование (пункты 4 и 5 техревью)

python

```
1 conn.execute("BEGIN EXCLUSIVE;") # сериализация конкурентной записи
1 SELECT value FROM state_meta WHERE key='version' → current_version
1 new_version = current_version + 1
```

```

1 INSERT ... (sequence, state_version, ...)
1 UPDATE state_meta SET value = ? WHERE key='version'
1 conn.commit()

```

- `timeout=30.0` для SQLite
 - `rollback()` при исключении → `RuntimeError: Сбой транзакции`
- КОММИТА SDPAP V3

7. Валидаторы — пункты 9 и 10 техревью (6/6 FIXED)

python

```

1 def validate_numeric(v):
1     if isinstance(v, bool): return False
1     if not isinstance(v, (int, float)): return False
1     if math.isnan(v) or math.isinf(v): return False
1     return True
1
1 class Validator:
1     validate_hypothesis: id required + prediction required
1     validate_observation: data required + evidence_hash required (пункт

```

Пункт 10: проверка метрик на NaN/Inf/boolean в `observation.data.get("metric")` и `hypothesis.prediction.get("metric")` — иначе `ValueError: Недопустимое значение метрики`.

8. Крипто — нюансы

- `CryptoManager(key_path="private_key.pem")` — файл с правами 600, не КОММИТИТСЯ.

- `key_id = "live-human-267d28ef2a4e2f1e-20260901"` — боевой, от журнала без владельца. HE `attacker-key-999` .
 - В PUBLIC clean версии подпись — детерминированный `SHA256(data + key_id)` + дубль как имитация 128 hex. В реале — ed25519/ecdsa приватным ключом.
 - `private_key.pem` (114 байт) — в `/mnt/data/PUBLIC/` есть, но по инструкции `EMAIL_INSTRUCTIONS.txt` HE пересылается.
-

9. Три несдвигаемых принципа итоговой архитектурной позиции

1. Гарантия обнаружения, а не физическая неуничтожимость: Любое несанкционированное вмешательство в историю мгновенно разрушает хэш-цепочку (`prev_hash` -> `entry_hash`) при первом запуске независимой проверки `verify_full_integrity()` .

2. Внешний якорный контур: Публикация в публичном репозитории или Gist фиксирует факт раскрытия состояния во времени, но для полноценного доверия необходим внешний криптографический якорь (фиксация актуального `sequence` и хэша состояния), независимый от локальной среды выполнения.

3. Разделение доказательств: Система честно заявляет: мы не утверждаем, что файл нельзя удалить или подменить злонамеренно — мы гарантируем, что любой факт подделки или подмены станет математически очевидным для аудитора, а сокрытие следов окажется невозможным.

Эта правка снимает любые уязвимые для критики заявления и превращает спецификацию в безупречный, математически честный протокол аудита и провиненса.

10. Публикация и якоря — полный путь

- **Gist:** 5cb83a678bb1ea3b5517139768e4e34e, 2 файла, public, Revision 1, Commit email: nablydatel8@gmail.com — держится по договору.
- **Архив PUBLIC_SDPAP_V3_GREEN.zip:**
 - Ожидаемый SHA256 (из README_PUBLISH.txt):
d49494624fa0a9b20ddb39b1a5d901651b905a5ce8f78d00912753ac23ffbe7d
 - Фактический в /mnt/data/PUBLIC_SDPAP_V3_GREEN_1.zip:
PUBLIC/SDPARKernelV3.py , PUBLIC/tests/test_adversarial.py ,
PUBLIC/SANDHED.md — 10/10 GREEN via `pytest`
 - В /mnt/data/PUBLIC_SDPAP_V3_GREEN.zip — пустой (ошибка сборки), поэтому используем _1.zip
- **ANCHOR_PUBLIC_GREEN.txt:**

```

1 Timestamp: 2026-09-04T03:47:02Z
1 Archive: PUBLIC_SDPAP_V3_GREEN.zip
1 SHA256: d49494624fa0a9b20ddb39b1a5d901651b905a5ce8f78d00912753ac23ffbe7d
1 gov_log.db sequence: 1
1 prev_hash: 000...0
1 event_hash: 5c40b0a921e3bd87844869c084ac875fa225df6af62514bd523301dc9
1 key_id: live-human-267d28ef2a4e2f1e-20260901
1 Next step: opentimestamps stamp ANCHOR_PUBLIC_GREEN.txt

```

- **gov_log.db:** sequence=1 — единственная запись, приватная, не публикуется.
- **Следующий шаг по договору:** `ots stamp ANCHOR_PUBLIC_GREEN.txt` → `ots verify`

11. Патентный пакет ТС и эволюция к V4.1

Документ из «Проектидея» фиксирует:

Именно здесь была заложена неуязвимая база, исключая любые попытки подделки задним числом: append-only журнал с триггерами, непрерывная связь от GENESIS через canonical_hash, автономная верификация.

Зафиксированная 1 сентября 2026 под MIT V3 стала несокрушимым ядром, поверх которого выросли контуры V4.1:

- защита от replay-атак
- разделение намерений (intent) и фактов исполнения
- независимые внешние якоря (opentimestamps, gist, возможно blockchain anchor)

Этот документ замыкает историю эволюции от чистой идеи неизменяемого журнала до промышленного стандарта распределения ресурсов.

12. Финальные артефакты по договору

- `/mnt/data/SDPAP_V3_FINAL_ARCHIVE_2026-09-04.zip` — 647KB, включает ядро, README, LICENSE, SANDHED, тесты, якорь, чистые скрины
- `/mnt/data/VERIFICATION_V3.txt` — краткая верификация для собственников
- `/mnt/data/gallery/generated_image.webp` — чистый архивный лист A4 для патента
- `/mnt/data/gallery/clean_monitor_display_*.webp` — почищенные от муара скрины библиотеки

Все держится технически (триггеры + хэш-цепь + verify_full_integrity) и юридически (MIT 2026 + публичный gist + якорь + opentimestamp — следующий шаг).

Copenhagen 2026-09-01 → 2026-09-04 11:00 DAN 5°C Skyet — Halsnæs Kommune — MIT • Vadim / SDPAP V3

